Proprietà della scelta greedy, ASP e Huffman

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 10, 2024
Studiato	Done

Caso studio: activity selection problem

Supponiamo di avere un certo numero di attività possibili. Ogni attività ha un tempo di inizio e un tempo di fine. Rappresentiamo tale insieme di attività come l'insieme:

$$S = \{ a_1, ..., a_n \}$$

Ogni attività a_i ha un tempo di inizio s_i e un tempo di terminazione f_i. Ossia, per ogni i:

$$a_i = [s_i, f_i)$$
$$s_i < f_i$$

L'obiettivo del problema è selezionare un sottoinsieme $A \subseteq S$ di attività compatibili di cardinalità massima. Due attività a_i, a_j sono compatibili se:

$$[s_i, f_i) \cap [s_j, f_j) = \emptyset$$

Il problema può essere risolto con l'approccio greedy.
Innanzitutto, ordiniamo le attività per tempo di terminazione crescente. La nostra scelta greedy è
l'attività con il più piccolo tempo di terminazione a^*. Eliminiamo allora da S tutte le attività che si sovrappongono ad a^* e a^* stesso (in quanto verrà incluso nella

soluzione A^*). Ripetiamo tale processo fino a quando non ci sono attività che si sovrappongono.

La complessità è $O(n)$ per la scelta greedy, e $O(n \log n)$ per l'ordinamento, quindi $O(n \log n)$. Dimostriamo la proprietà greedy.

> *Sia S_k un sottoproblema non vuoto e sia a_m l'attività di S_k con il più piccolo tempo di terminazione. Allora a_m è inclusa in qualche soluzione ottima, ovvero in qualche sottoinsieme di attività compatibili di S_k di cardinalità massima.*

Dimostrazione

Sia A_k una soluzione ottima, ossia un sottoinsieme di cardinalità massima di attività compatibili. Chiamiamo a_j l'attività che termina prima in A_k. Abbiamo due casi:

- se $a_j = a_m$ allora abbiamo finito. Infatti avremo che a_m è in A_k e A_k è una soluzione ottima quindi la proposizione della tesi è dimostrata;
- se $a_j \neq a_m$, allora $a_m \notin A_k$. Quindi posso rimuovere a_j da A_k e inserire a_m. Tale insieme è costituito da attività compatibili, infatti a_m ha il più piccolo tempo di terminazione. Formalmente possiamo dire che:

$$
\begin{gathered}
A_k \setminus \{ a_j \} \cap \{ a_j \} = \emptyset \\
A_k \setminus \{ a_j \} \cap \{ a_m \} = \emptyset \\
\text{in quanto } a_m = [s_m, f_m) \text{ e } f_m \leq f_j
\end{gathered}
$$

  L'insieme che si ottiene, inoltre, ha la stessa cardinalità di A_k in quanto abbiamo solamente sostituito un elemento con un altro. Quindi abbiamo un insieme di cardinalità massima in cui vi è l'elemento a_m.

Dimostrazione della proprietà di scelta greedy per l'algoritmo di Huffman

L'algoritmo di Huffman costruisce un albero di codifica prefix-free le cui foglie sono i caratteri dell'alfabeto del testo da codificare. Si interpreta la codifica di un carattere come il cammino semplice dalla radice alla foglia che rappresenta tale

carattere. Nell'albero si interpreta con 0 la discesa verso il figlio sinistro, 1 la discesa verso il figlio destro.

L'albero ha $|C|$ foglie dove C è l'alfabeto di caratteri da codificare. Ha, inoltre, esattamente $|C| - 1$ nodi interni.

Indicheremo tale albero con la lettera T. Utilizzeremo $c.freq$ per indicare la frequenza del carattere c, e chiamiamo $d_T(c)$ come la profondità di c nell'albero T , ossia la lunghezza della sua codifica, cioè il numero di bit necessari per rappresentarlo. Il costo dell'albero è il numero di bit per codificare il testo, ed è definibile come:

$$B(T) = \sum_{c \in C} d_T(c) \cdot c.freq$$

La frequenza $c.freq$ non varia, tuttavia d_T varia in virtù dell'albero T. Tra tutti i costi $B(T)$, vogliamo scegliere il più piccolo. Dobbiamo quindi scegliere T tale che $B(T)$ è minimo.
La strategia greedy consiste nello
scegliere i caratteri con la frequenza minima. Dobbiamo dimostrare che i caratteri che sono più frequenti utilizzano il minor numero di bit e il viceversa. Ossia, i caratteri più frequenti avranno un d_T più piccolo, e i caratteri meno frequenti avranno un d_T più alto.

> *Sia C un alfabeto e consideriamo un testo i cui caratteri sono in C e in cui un carattere c ha frequenza $c.freq$. Siano $x, y \in C$ i caratteri con le frequenze minime. Allora esiste un codice prefix-free ottimo in cui le codifiche di x e y hanno la stessa lunghezza massima e differiscono per l'ultimo bit.*

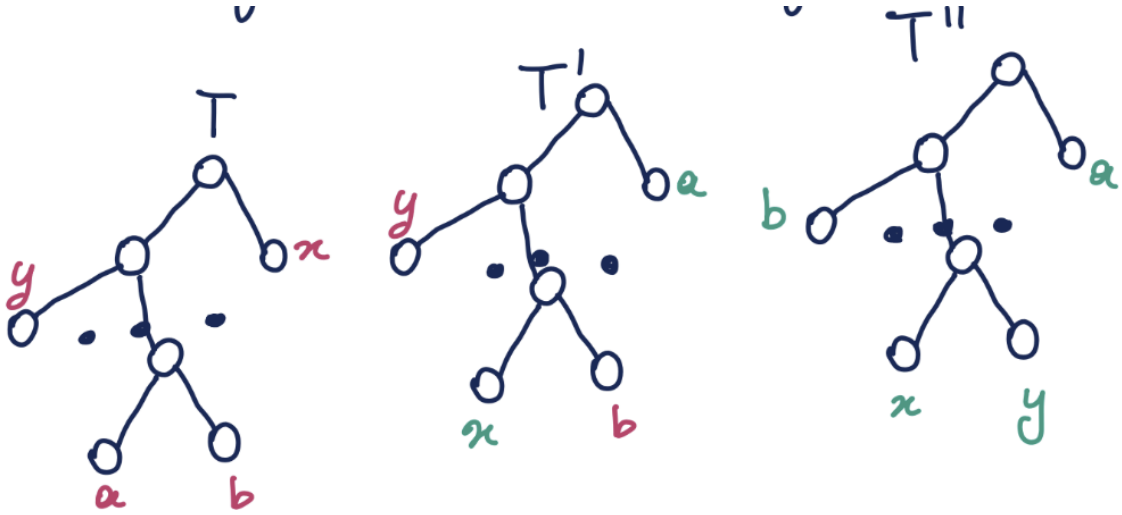
Con "stessa lunghezza massima" si indica il fatto che si trovano in profondità massima.

Sia T un albero che rappresenta un codice di prefix-free ottimo, quindi segue che il suo costo $B(T)$ è minimo. Siano a e b i due caratteri che corrispondono alle foglie di massima profondità. Senza ledere generalità al discorso, possiamo

assumere che $a.freq \leq b.freq$. Inoltre $x.freq \leq y.freq$. Avremo quindi che $x.freq \leq a.freq$ e $y.freq \leq b.freq$. Questo perché, per ipotesi, x e y hanno frequenza minima.

Può accadere che $x.freq = a.freq$ e $y.freq = b.freq$. Ma se $x.freq = b.freq$ allora tutte le frequenze sono uguali, quindi $a.freq = b.freq = x.freq = y.freq$ e la proposizione è banalmente verificata: x e y hanno infatti la stessa lunghezza massima. Perciò possiamo assumere che $x.freq < b.freq$ e quindi $x \neq b$.

Nell'albero T possiamo scambiare a con x ottenendo l'albero T'. In T' scambio b con y. In questo modo otteniamo T'' in cui x e y hanno profondità massima:



Osserviamo che se $x = b$ e $y \neq a$ allora T'' non avrebbe x e y a profondità massima.

Dimostriamo che T'' così ottenuto è tale che $B(T'') = B(T)$. Per fare ciò, basta provare che $B(T'') \leq B(T)$, e siccome $B(T)$ è minimo, allora $B(T'')$ è minimo. Iniziamo da
$B(T')$ e mostriamo che $B(T') \leq B(T)$:

$$B(T) - B(T') = \sum_{c \in C} c.freq \cdot d_T(c) - \sum_{c \in C} c.freq \cdot d_{T'}(c) =$$
$$= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a) =$$
$$= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq.d_T(a) - a.freq \cdot d_T(x) =$$
$$= (a.freq - x.freq) \cdot (d_T(a) - d_T(x)) \geq 0$$

poiché $d_T(c) = d_{T'}(c)$ per ogni $c \neq a \wedge c \neq x$. Scambiando le lettere, si scambia la profondità, quindi $d_{T'}(a) = d_T(x)$ e $d_{T'}(x) = d_T(a)$. Inoltre, x ha frequenza minima e $d_T(a) \geq d_T(x)$. Ne deduciamo quindi che $B(T) - B(T') \geq 0 \iff B(T) \geq B(T')$.

Analogamente si dimostra che $B(T'') \leq B(T')$, e per proprietà transitiva si deduce $B(T'') \leq B(T)$.

Dimostrazione delle proprietà di scelta greedy e di sottostruttura ottima nelle versioni del problema dello zaino

Ricordiamo la definizione del problema dello zaino. Abbiamo n oggetti per cui sono specificati valore v e peso w. Definiamo W come il peso massimo. L'obiettivo del problema è scegliere un sottoinsieme S degli oggetti tale che il valore complessivo degli oggetti è massimo e il peso complessivo è minore di W. Ossia:

$$\sum_{i \in S} w_i \leq W \wedge \max \sum_{i \in S} v_i$$

Questo problema può essere riscritto come il problema di massimizzare:

$$\sum_{i=1}^{m} x_i \cdot v_i$$

Dove $x_i = 1$ se prendiamo l'oggetto x_i, 0 se non scegliamo tale oggetto. Quindi il problema riguarda l'**assegnamento per le variabili** x_i. Vogliamo inoltre che:

$$\sum_{i=1}^{m} x_i \cdot w_i \leq W$$

Tuttavia, **come cambia il problema se** x_i **non è un valore binario, bensì un numero reale in** $[0, 1]$**?**